BOOKEND | UML CLASS DIAGRAM

Library lending system · Core business objects and the records that persist them

CORE DOMAIN | Rules and concepts the application works with

«abstract» User

Shared account rules and identity. Abstract: users sign in as a specific role.

user_id · email · username · active

check_pass() · activate() · deactivate()

Patron

Library member who borrows and saves books.

Inherits account details
Has an active-loan limit

borrow · return · review · favorite

Librarian

Staff account for catalog and library operations.

Inherits account details
Manages members and loans

catalog · fees · genres · notices

Book

A catalog title with tracked physical copies and availability.

book_id · title · author · genre
total · available · on-loan · reserved copies

borrow() · return_book() · archive()

NewReleaseBook

A book with a shorter lending period.

Inherits book details
Loan period: 7 days

Uses standard borrow / return rules

ReferenceBook

A book available for reading in the library only.

Inherits book details
Not borrowable for take-home use

can_borrow() → no

Loan

One borrowing event connecting a member to a book.

loan_id · borrowed_at · due_date · returned_at
patron_id → Patron · book_id → Book

mark_returned() · mark_lost()
outstanding() · fee_breakdown()

Reservation

A place in the queue when all borrowable copies are in use.

patron_id → Patron · book_id → Book
status: waiting · ready · fulfilled · cancelled

Waiting reservations promote FIFO

«abstract» Fee

A charge associated with a loan. Abstract fee calculation.

paid · amount(loan, current_date)

outstanding()

LateFee

Accumulates when a book is returned after its due date.

Calculated per day overdue
Waived for a lost book

amount(loan, date)

LossFee

Fixed charge when a loan is marked as a lost book.

Fixed amount
Recorded on the loan

amount(loan, date)

HOW THE MAIN RELATIONSHIPS WORK

1 Patron → 0..* Loans
A member can have multiple loan records.

1 Book → 0..* Loans
A title can appear in many loans over time.

1 Patron → 0..* Reservations
A member may queue for several titles.

1 Book → 0..* Reservations
Reservations queue by book, first in first out.

1 Loan ◆ 0..2 Fee types
A loan may have a late fee and/or loss fee.

DATABASE RECORDS | SQLAlchemy classes that store the application data

Repositories translate between these stored records and the domain objects above. Record classes are not domain subclasses.

UserRecord

Table: users
Accounts: email, username, role, active status.

BookRecord

Table: books
Catalog details, book kind, copy count, archive state.

LoanRecord

Table: loans
Borrowing dates, member/title IDs, return and lost state.

ReservationRecord

Table: reservations
Member/title IDs, queue status, created time.

GenreRecord

Table: genres
Unique genre names used to organize catalog books.

FavoriteRecord

Table: favorites
Saved member-to-book pairs.

BasketRecord

Table: basket_items
Books a member has placed in their borrow basket.

ReviewRecord

Table: reviews
Member rating and comment for a book.

FeePaymentRecord

Table: fee_payments
Payment, receipt and confirmation details for a loan.

NotificationRecord

Table: notifications
In-app messages, read state and recipient.

UML key —△— Inheritance (open triangle points to the parent class) ◆ Composition: fee types belong to a loan. 1 / 0..* One-to-many relationship counts.

Scope: core domain classes and the main persistence records. Routes, services and repositories are omitted to keep the model readable.

Based on Bookend-LMS backend/app/models and backend/app/repositories.